

The above command is for cleaning the document and database.

Using CouchDBKit

The goal of CouchDBKit is to provide a dedicated framework for the Python application to manage and access CouchDB.

The following features are inherent with CouchDBKit:

- To use the *http* backend using *py-restclient*
- Managing documents dynamically
- Threadsafe
- Attaching design docs with the application and sending these to CouchDB
- Managing documents with a dynamic schema

In order to install CouchDBKit using Pip, use the following commands:

```
$ curl -O http://python-distribute.org/distribute_setup.py
$ sudo python distribute_setup.py
$ easy_install pip
```

For installation or upgrading to the latest released version of CouchDBKit, use the command given below:

```
$ pip install couchdbkit
```

The following is the code you can use to work with CouchDBKit:

```
from couchdbkit import Server
myserver = Server()
db = myserver.create_db("couchdbkit_test")
db['myid'] = { 'x': 'Hello' }
doc = db['myid']
```

You can map a CouchDB object to a Python object easily with a dynamic schema, as follows:

```
from couchdbkit import Document
class MyDBClass(Document):
    author = StringProperty()
    content = StringProperty()
    date = DateTimeProperty()
MyDBClass = MyDBClass()
MyDBClass.author = "AuthorName"
MyDBClass.homepage = "http://couchdbkit.org"
```

Once this is done, the first CouchDB document will be as shown below:

```
import datetime
from couchdbkit import *
class MyDBClass(Document):
    author = StringProperty()
    content = StringProperty()
```

```
date = DateTimeProperty()
```

Here is the code to save a MyDBClass on the 'MyDBClass' database. We also see how to create a database.

```
server = Server()
db = server.get_or_create_db("MyDBClass")
MyDBClass.set_db(db)
MyDBClass = MyDBClass
(
    author="AuthorName",
    content="Welcome",
    date=datetime.datetime.utcnow()
) MyDBClass.save()
```

Programming Python – MongoDB

First, MongoDB and PyMongo are installed so that the connection of Python with MongoDB can be established. In the Python shell, the following instruction is executed:

```
$ import pymongo
```

Next, the MongoClient is created by running the *mongod* instance. The following code is connected on the default host and port:

```
>>> from pymongo import MongoClient
>>> myclient = MongoClient()
```

The specific host and port can be mentioned explicitly as follows:

```
>>> myclient = MongoClient('localhost', 27017)
```

Alternatively, the MongoDB URI format can be used as shown below:

```
>>> myclient = MongoClient('mongodb://localhost:27017')
```

The MongoDB instance is able to support multiple and independent databases, which can be obtained with the use of attribute style access on MongoClient instances, as follows:

```
>>> mydb = myclient.MyTestDatabase
```

Similarly, other instructions of Python for MongoDB can be used for unstructured and Big Data processing. 

By: Dr Gaurav Kumar

The author is the MD of Magma Research and Consultancy Pvt Ltd. He is associated with a number of academic institutes, where he delivers lectures and conducts technical workshops on the latest technologies and tools. He can be contacted at kumargaurav.in@gmail.com